Below is a **structured reference table of commonly used classes in the java.util package**, followed by a **detailed table of the Arrays class methods and how they collaborate with other utilities like Streams, Collections, and Comparators.**

---

1. Commonly Used Classes in java.util Package

Class	Category	Common Methods	Purpose / Usage
ArrayList	List	add(), remove(), get(), set(), size()	Dynamic resizable array implementation of List
LinkedList	List / Deque	addFirst(), addLast(), removeFirst(), removeLast()	Doubly linked list implementation
Vector	List	add(), get(), remove(), capacity()	Thread-safe dynamic array (legacy)
Stack	LIFO	push(), pop(), peek(), empty()	Stack implementation
HashSet	Set	add(), remove(), contains()	Stores unique elements using hash table
LinkedHashSet	Set	add(), remove(), iterator()	Maintains insertion order
TreeSet	Sorted Set	add(), first(), last(), higher()	Sorted set using Red-Black tree
HashMap	Map	put(), get(), remove(), containsKey()	Key-value storage using hashing
LinkedHashMap	Map	put(), get(), remove()	Maintains insertion order
TreeMap	Sorted Map	put(), firstKey(), lastKey()	Sorted map implementation
Hashtable	Map	put(), get(), contains()	Thread-safe legacy map
PriorityQueue	Queue	add(), poll(), peek()	Priority-based queue

ArrayDeque	Queue / Deque	addFirst(), addLast(), removeFirst()	Resizable array-based deque
Collections	Utility	sort(), reverse(), shuffle(), min()	Utility methods for collections
Arrays	Utility	sort(), binarySearch(), copyOf()	Utility methods for arrays
Objects	Utility	equals(), hash(), requireNonNull()	Object utility methods
Optional	Container	isPresent(), get(), orElse()	Avoids NullPointerException
Scanner	Input Utility	nextInt(), nextLine(), nextDouble()	Reads input from console/file
Random	Utility	nextInt(), nextDouble()	Random number generation
UUID	Utility	randomUUID(), toString()	Unique identifier generation
Date	Date/Time	getTime(), before(), after()	Legacy date representation
Calendar	Date/Time	get(), set(), add()	Date manipulation
Timer	Scheduling	schedule()	Task scheduling
TimerTask	Scheduling	run()	Task executed by Timer

---

2. java.util.Arrays Class Overview

Arrays is a **utility class that provides static methods to manipulate arrays.**

Common uses:

- Sorting arrays
- Searching arrays
- Converting arrays to collections
- Filling arrays
- Comparing arrays

- Streaming arrays

---

3. Arrays Class Methods and Collaborations

Method	Syntax Example	Collaboration With	Purpose
sort()	Arrays.sort(arr)	Comparable / Comparator	Sorts array elements
parallelSort()	Arrays.parallelSort(arr)	ForkJoinPool	Parallel sorting for large arrays
binarySearch()	Arrays.binarySearch(arr, key)	Sorted arrays	Fast searching using binary search
equals()	Arrays.equals(arr1, arr2)	Object.equals()	Compares arrays
deepEquals()	Arrays.deepEquals(a,b)	Multidimensional arrays	Deep comparison
fill()	Arrays.fill(arr, value)	Primitive arrays	Assigns value to all elements
copyOf()	Arrays.copyOf(arr, newSize)	Memory management	Creates resized copy
copyOfRange()	Arrays.copyOfRange(arr, from, to)	Array slicing	Copies specific range
toString()	Arrays.toString(arr)	String class	Converts array to readable string
deepToString()	Arrays.deepToString(arr)	Nested arrays	Prints multidimensional arrays
asList()	Arrays.asList(arr)	List interface	Converts array to List
stream()	Arrays.stream(arr)	Java Streams API	Converts array to Stream
setAll()	Arrays.setAll(arr, generator)	Lambda expressions	Initialize array dynamically
parallelPrefix()	Arrays.parallelPrefix(arr,	Functional interface	Cumulative

	op)		operations

---

4. Arrays Collaboration Examples

1. Arrays + Comparator (Custom Sorting)

```
import java.util.Arrays;
import java.util.Comparator;

public class Example {
   public static void main(String[] args) {
      String[] names = {"Rahul", "Ankit", "Vijay"};

      Arrays.sort(names, Comparator.reverseOrder());

      System.out.println(Arrays.toString(names));
   }
}
```

---

2. Arrays + Streams

```
import java.util.Arrays;

public class Example {
   public static void main(String[] args) {

      int[] numbers = {10,20,30,40};

      int sum = Arrays.stream(numbers).sum();

      System.out.println(sum);
   }
}
```

---

3. Arrays + Collections

```
import java.util.*;

public class Example {

   public static void main(String[] args) {

      String[] arr = {"A","B","C"};

      List<String> list = Arrays.asList(arr);
```

```
        Collections.sort(list);

        System.out.println(list);
    }
}
```

---

5. Real-World Use Cases of Arrays Class

Scenario	Arrays Method Used	Explanation
Sorting student marks	Arrays.sort()	Sort marks ascending
Searching employee ID	Arrays.binarySearch()	Efficient search
Initialize array with default values	Arrays.fill()	Fill array quickly
Convert array to list	Arrays.asList()	Used with Collections API
Stream processing	Arrays.stream()	Used with Java Streams
Copy array safely	Arrays.copyOf()	Prevent mutation

✅ **Key Concept**

Arrays acts as a **bridge between primitive arrays and modern Java APIs** like:

- **Collections Framework**
- **Streams API**
- **Comparator / Comparable**
- **Functional Interfaces**

If you want, I can also show a **complete comparison table of Arrays vs Collections vs Stream utilities in Java (very important for interviews and Java architecture understanding).**